Introduction to LLD.

Shubham Chaudhari

@ Nokia Lead Software engineer
DSA, LLD, HLD.

Agenda :

① What is Low level System Design
② Difference between HLD & LLD
③ Standard coding guidelines.
④ Basic Building Blocks
⑤ OOPs
⑥ UML.

① What is LLD ?

- focuses on internal working of individual components.
- defines how classes, methods, datastructure, and interations will be implemented.
- Imp for SDE.

② Difference between LLD & HLD.

HLD
- Provides major overview of components and their interaction.
- Techonology choices
  - database
  - platform services.
  - etc.
- How our system will meet the functional requirements.
- Diagram and Documents.
  - Block Diagram
  - interaction diagram
  - capacity planing.

③ LLD vs HLD.

① HLD explains architecture.
: services, components, database, caching, messaging Queues.

LLD explains implementation
: class names, methods names, object interaction.

② HLD defines what needs to be build.
LLD defines How it should be built but at code level.

③ HLD includes diagrams like Sys architecture, data floc.....
LLD includes class diagrams, methods, attributes, relationships, etc.

③ Standard Coding guidelines.

- meaningful variable names.

~~Status - T/F~~
~~bookstatus - T/F~~
~~bookingstatus - T/F~~

✓ isBooked → False.
— True.

mm ( )
{
}
mm ( ) {
}

ABbc DaaEaa.

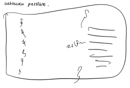
✓ getDriverinfo.
└ use camel case
└ constant - uppercase.

* Indentation style.
- use spaces or tabs (4 spaces).
- Align code properly, to increase readability.

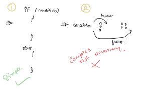
★ Programming Principles/ Ideologies.

1. DRY - "Don't Repeat yourself" :

To avoid code duplicacy, create functions whenever possible.

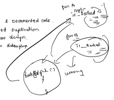


KISS - keep is Simple, stupid.



- write Simple logic
- complex soln should be used when necessary.

Benefits :
- Clear & Documented code.
- Reduced Duplication.
- modular design.
- Easier debugging.



★ Building Blocks of LLD.

OOP - object oriented programming.

class - Blueprint of Object (user defined datatype).

- members/ (variables / constants)
- functions / methods.

A. getiban— —

```
class BankAccount {
    Private :
        int m_accountNO;
        double m_balance;

    Public :
        string m_IFSC

    Private :
        get_Balance (int accountno)
        {
            return m_balance.
        }
    Public :
        Showbalance. ( )
}
```

encapsulation. the data & function.

Object : instance of class.

- BankAccount MyAccount = new BankAccount ("14834").

BankAccount → classname → datatype.
MyAccount → objectname → variable.

★ Major elements of OOPS :

└ Abstraction - Hide unnecessary implementation Details.
└ Encapsulation - Protect internal state of object (wrap member & member)
└ inheritance - Reuse code across related classes.
└ Polymorphism. many forms. - Use one interface for multiple behaviours.

Abstraction :

```
abstract class Shape
{
    abstract void draw ( );
}
```

If class is defined as abstract, then I can not create object of that.

All the subclass should implement this methode,

✓ class circle ( ) { draw ( ); }
✓ class square ( ) draw ( );

Encapsulation : Bundling (data & function)
⇓
class

Access specifier → who can access the function & data.

- Private
- Public
- Protected.

Inheritance

It allows to inherit data members & methods to the child classes.

BankAccount - Parent class.
- acc no
- Balance
- IFSC.
- deposit
- withdraw
- checkBalance.

Saving AC (Pays Quaterly interest)
Current AC. (do not pay Quaterly interest).

Polymorphism. : one interface multiple behaviour.

```
- add (int a, int b)
  {
    return a+b;
  }                          → ①

- add (double a, double b)
  {
    return a + b;
  }                          → ②

add (1, 2)
add(1.5 , 2.3)
```

Upload (Pdf)
(doc)
(CSV).